



Java Cheat Sheet

Last Updated : 9 Oct, 2025

Java is a high-level, object-oriented programming language developed by James Gosling in 1991. It can run on any operating system and follows the Write Once, Run Anywhere (WORA) principle. Java's key features, known as Java Buzzwords, include:

- **Platform-independent:** runs on any OS with a JVM
- **Object-oriented:** based on classes and objects

This **Java Cheat Sheet** article has been written by experts in Java and is based on the experience of students who have recently undergone Java interviews.

1. Java Programming Terminologies

- **JVM:** executes the bytecode generated by the compiler.
- **Bytecode:** The Javac compiler of JDK compiles the Java source code into bytecode so that it can be executed by JVM.
- **JDK:** It is a complete Java development kit that includes everything including compiler, Java Runtime Environment (JRE), java debuggers, java docs, etc.
- **JRE:** allows the Java program to run, however, we cannot compile it.
- **Garbage Collector:** To delete or recollect that memory JVM has a program called Garbage Collector.
- **Finalize method:** this function is triggered by the garbage collector just before an object is deleted or destroyed.

2. Java Basics

Now, we will explore some of the fundamental concepts often utilized in the Java programming language.



Object

An object refers to an entity that possesses both behavior and state, such as a bike, chair, pen, marker, table, and car. These objects can be either tangible or intangible, including the financial system as an example of an intangible object.

There are three characteristics of an object:

- **State:** The data (value) of an object is represented by its state.
- **Behaviour:** The functionality of an object, such as deposit, withdrawal, and so on, is represented by the term behaviour.
- **Identity:** A unique ID is often used to represent an object's identification. The value of the ID is hidden from the outside user. The JVM uses it internally to uniquely identify each object.

Class

A class is a collection of objects with similar attributes. It's a blueprint or template from which objects are made. It's a logical thing. It can't be physical. In Java, a class definition can have the following elements:

- **Modifiers:** A class can be private or public, or it can also have a default access level
- **class keyword:** To construct a class, we use the class keyword.
- **class name:** The name of the class should usually start with a capital letter.
- **Superclass (optional):** If the class has any superclass, we use the extends keyword and we mention the name of the superclass after the class name.
- **Interface (optional):** If the class implements an interface, we use the implements keyword followed by the name of the interface after the class name.

Constructors

In Java, a constructor is a block of code similar to a method. Whenever a new class instance is created, the constructor is called. The memory allocation for the object only happens when the constructor is invoked.

There are two types of constructors in Java. They are as follows:

1. **Default Constructor:** A default constructor is a type of constructor that does not require any parameters. When we do not declare a constructor for a class, the compiler automatically generates a default constructor for the class with no arguments.
2. **Parameterised Constructor:** A parameterized constructor is a type of constructor that requires parameters. It is used to assign custom values to a class's fields during initialization.

Keyword

In Java, *Reserved words* are also known as keywords. These are particular terms that hold specific meanings. Java has 61 Reserved Keywords that are predefined and cannot be used as variable, object, or class names. Here's a list of the keywords used in Java:

To know more about java keyword -> [Java keyword](#)

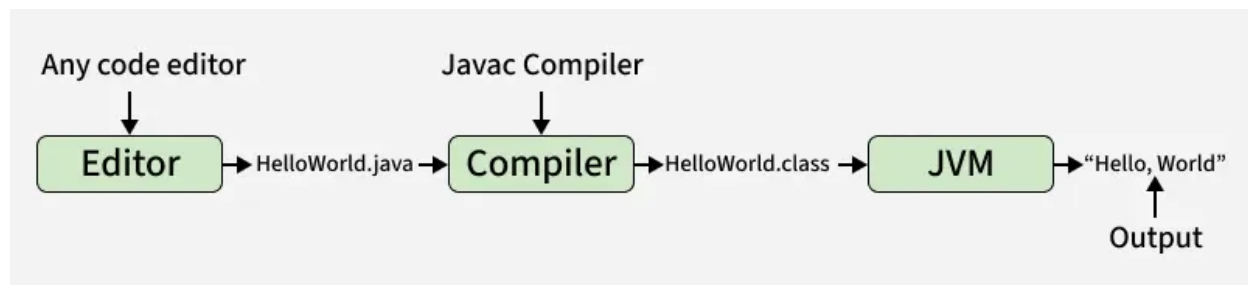
3. Java Program to Print "Hello World"

```
class GFG {  
  
    public static void main(String[] args)  
    {  
  
        System.out.println("Hello World!");  
    }  
}
```

Output

Hello World!

How Program Work:



4. Data Types in Java

Data Types in Java are the different values and sizes that can be stored in the variable according to the requirements.

Java Datatype are further two types:

1. Primitive Data Type in Java

In Java, primitive data types serve as the foundation for manipulating data. They are the most basic types of data that the Java programming language uses. Java has several primitive data types, including: byte, short, int, long, float, double, char and boolean

2. Non-primitive Data Type

Non-primitive datatypes are created from primitive datatypes. Examples of non-primitive datatypes include Arrays, String, and Classes.

To know more about Data Types-> [Data Types](#)

5. Java Comments

There are three types of comments in Java

1. Single Line comment

To comment on a single line of code, you can use a double forward slash "//" followed by your message. This syntax is commonly used for coding.

```
import java.io.*;

class GFG {
    public static void main(String[] args)
    {
        System.out.println("GFG!");
        // System.out.println("This line is not executed");
    }
}
```

Output

GFG!

2. Multi-line comment

If you need to comment on multiple lines of code, you can utilize the syntax of a double forward slash "/*". Simply enter your message between the two symbols, and complete the comment with "*/". This is a widely used syntax in coding.

```
/*package whatever //do not write package name here */

import java.io.*;

class GFG {
    public static void main(String[] args)
    {
        System.out.println("GFG!");
    }
}
```

Output

GFG!

3. JavaDoc Type Comment

When working on a project or software package, it is often helpful to use this method as it can assist in creating a documentation page for reference. This page can provide information on available methods, their parameters, and more, making it a useful tool for learning about the project.

Syntax:

```
/** Comment starts  
*This is  
*sample comment */
```

6. Java Variables

Variables are the containers that save the data values. Each variable is assigned according to the data type it is assigned.

Syntax:

```
data_type var_name;
```

Types of Variables

There are 3 types of Data types in Java as mentioned below:

- **Local Variables:** A variable defined within a block or method or constructor is called a local variable.
- **Instance Variables:** Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.
- **Static Variables:** Static variables are also known as class variables. The static variables are declared using the static keyword within a class outside of any method, constructor, or block.

```
class GFG {  
    public static void main(String[] args)  
    {  
  
        // Integer variable  
        int age = 25;  
  
        // Floating-point variable  
        float price = 99.99f;  
  
        // Character variable  
        char grade = 'A';  
  
        // Boolean variable  
        boolean isJavaFun = true;  
  
        // Display values  
        System.out.println("Age: " + age);  
        System.out.println("Price: " + price);  
        System.out.println("Grade: " + grade);  
        System.out.println("Is Java fun? " + isJavaFun);  
    }  
}
```

}

.

Age: 25
 Price: 99.99
 Grade: A
 Is Java fun? true

7. Access Modifiers in Java

Access modifiers help to restrict the scope of a class, constructor, variable, method, or data member. It provides security, accessibility, etc to the user depending upon the access modifier used with the element

Types of Access Modifiers in Java

There are four types of access modifiers available in Java:

- **Default**
- **Private**
- **Protected**
- **Public**

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

Access modifier

8. Operators in Java

A Java [operator](#) is a symbol that performs a specific operation on one or more operands (variables or values) in an expression.

Precedence and Associativity of Operator in Java

Here is the table representing the precedence order of Java operators from high to low as we move from top to bottom.

Operators	Associativity	Type
++, -	Right to Left	Unary postfix

Operators	Associativity	Type
++, --, +, -, !	Right to Left	Unary prefix
/, *, %	Left to Right	Multiplicative
+, -	Left to Right	Additive
<, <=, >, >=	Left to Right	Relational
==, !=	Left to Right	Equality
&	Left to Right	Boolean Logical AND
^	Left to Right	Boolean Logical Exclusive OR
	Left to Right	Boolean Logical Inclusive OR
&&	Left to Right	Conditional AND
	Left to Right	Conditional OR
?:	Right to Left	Conditional
=, +=, -=, *=, /=, %=	Right to Left	Assignment

9. Identifiers in Java

The name that we give to the class, variable, and methods are formally called identifiers in Java. and for defining Identifier there are some rules of it we need to take care of that while defining Identifiers.

Rules of defining Java identifiers:-

- Can contain letters (A-Z, a-z), digits (0-9), \$, and _.
- Cannot start with a digit.
- Case-sensitive (myVar ≠ MyVar).
- Cannot use Java reserved words (like while, int).
- No strict length limit, but short, meaningful names are recommended.
- Special characters (e.g., @, #) are not allowed.

10. Control Flow in Java

1. If, else-if, else

```
import java.io.*;

// Driver Class
class GFG {
    // main function
    public static void main(String[] args)
    {

        int a = 1, b = 2;

        if (a < b)
            System.out.println(b);
        else if (a > b)
            System.out.println(a);
        else
            System.out.println(a + "==" + b);
    }
}
```

Output

2

2. Nested if - else

A nested if is an if statement that is the target of another if or else. Nested if statements mean an if statement inside an if statement.

```
import java.util.*;

class NestedIfDemo {
    public static void main(String args[])
    {

        int i = 1;
        if (i == 10 || i < 15) {

            // First if statement
            if (i < 15)
                System.out.println("i is smaller than 15");

            // Nested - if statement
            // Will only be executed if statement above
            // it is true
            if (i < 12)
                System.out.println(
                    "i is smaller than 12 too");
        }
        else {
            System.out.println("i is greater than 15");
        }
    }
}
```

```
}
```

Output

```
i is smaller than 15  
i is smaller than 12 too
```

2. Switch Statement

A switch statement in Java is used to execute one block of code from multiple options based on the value of a variable or expression.

```
public class GFG {  
  
    // main driver method  
    public static void main(String[] args)  
    {  
  
        int day = 5;  
        String dayString;  
  
        // Switch statement with int data type  
        switch (day) {  
  
            // Case  
            case 1:  
                dayString = "Monday";  
                break;  
  
            // Case  
            case 2:  
                dayString = "Tuesday";  
                break;  
  
            // Case  
            case 3:  
                dayString = "Wednesday";  
                break;  
  
            // Case  
            case 4:  
                dayString = "Thursday";  
                break;  
  
            // Case  
            case 5:  
                dayString = "Friday";  
                break;  
  
            // Case  
            case 6:  
                dayString = "Saturday";  
                break;  
        }  
    }  
}
```

```
// Case
case 7:
    dayString = "Sunday";
    break;

// Default case
default:
    dayString = "Invalid day";
}

System.out.println(dayString);
}
}
```

Output

Friday

Loops in Java

Loops are used for performing the same task multiple times. There are certain loops available in Java as mentioned below:

1. For Loop
2. While Loop
3. do-while

Java Java Java

```
// Java Program to implement
// For loop
import java.io.*;

// Driver Class
class GFG {
    // Main function
    public static void main(String[] args)
    {
        int n = 5;
        for (int i = 1; i <= n; i++) {
            System.out.println(i);
        }
    }
}
```

Output

1
2
3

4
5

11. Methods in Java

Java Methods are collections of statements that perform some specific task and return the result.

Syntax of Method

```
<access_modifier> <return_type> <method_name> ( list_of_parameters )  
{  
    //body  
}
```

Below is the implementation of the above method:

```
import java.io.*;  
  
// Driver Class  
class GFG {  
    public static int sum(int i, int j) { return i + j; }  
  
    // main method  
    public static void main(String[] args)  
    {  
  
        int n = 3, m = 3;  
  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < m; j++) {  
  
                System.out.print(sum(i, j) + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
0 1 2  
1 2 3  
2 3 4
```

12. Java Input-output (I/O operations)

We can only print using System.out but can use different print varieties in it:

1. print: The cursor remains on the same line

```
System.out.print(" GFG " + x);
```

2. **println**: Cursor Moves to a new line

```
System.out.println(" GFG " + x);
```

3. **printf**: The cursor remains on the same line

```
System.out.printf(" GFG %d",x);
```

Formatted Print

```
System.out.printf("%7.5f", Math.PI);
```

Explanation of the above statement:

*7 is the Field width and .5 is the precision fo the floating number printed . So, answer will be **3.14159***

Taking Input in Java

1. Parsing Command-line arguments

We can take input using the Command line simp

```
class GFG {  
    public static void main(String args[])  
    {  
  
        for (int i = 0; i < args.length; i++)  
            System.out.println(args[i]);  
    }  
}
```

```
javac GFG.java
```

```
java GFG This is just to check 2
```

Output:

```
This  
is  
just  
to  
Check  
2
```

2. Buffer Reader Class

`InputStreamReader()` is a function that converts the input stream of bytes into a stream of characters so that it can be read as `BufferedReader` expects a stream of characters.

```
import java.io.*;

class GFG {

    // Main Method
    public static void main(String[] args)
        throws IOException
    {

        // Creating BufferedReader Object
        // InputStreamReader converts bytes to
        // stream of character
        BufferedReader bfn = new BufferedReader(
            new InputStreamReader(System.in));

        // String reading internally
        String str = bfn.readLine();

        // Integer reading internally
        int it = Integer.parseInt(bfn.readLine());

        // Printing String
        System.out.println("Entered String : " + str);

        // Printing Integer
        System.out.println("Entered Integer : " + it);
    }
}
```

Output:

```
ABC
11
Entered String : ABC
Entered Integer : 11
```

3. Scanner Class

Syntax:

```
Scanner scn = new Scanner(System.in);
```

```
import java.io.*;
import java.util.Scanner;

class GFG {
    public static void main(String[] args)
    {

        // creating the instance of class Scanner
        Scanner obj = new Scanner(System.in);
        String name;
        int rollno;
```

```
float marks;

// taking string input
System.out.println("Enter your name");
name = obj.nextLine();

// taking float input
System.out.println("Enter your marks");
marks = obj.nextFloat();

// printing the output
System.out.println("Name :" + name);
System.out.println("Marks :" + marks);
    }
}
```

Output:

```
ABC
65
Name :ABC
Marks :65
```

13. Java Polymorphism

Polymorphism is the ability of a single method or object to take multiple forms, allowing one interface to be used for different behaviors.

Types of Polymorphism:

1. **Compile-time (Static) Polymorphism:** Achieved through method overloading – same method name, different parameters.
2. **Runtime (Dynamic) Polymorphism:** Achieved through method overriding – subclass provides a specific implementation of a superclass method.

```
import java.io.*;

// Class
class ABC {

    // Sum Method with int returning value
    public int sum(int x, int y) { return x + y; }

    // Sum Method with float returning value
    public double sum(double x, double y) { return x + y; }
}

// Driver Class
class GFG {
    // main function
    public static void main(String[] args)
    {
```

```

        ABC temp = new ABC();

        System.out.println(temp.sum(1, 2));
        System.out.println(temp.sum(3.14, 4.23));
    }
}

```

Output

```

3
7.370000000000001

```

14. Java Inheritance

It is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class.

```

import java.io.*;

// Base or Super Class
class Employee{

    int salary = 70000;
}

// Inherited or Sub Class
class Engineer extends Employee{

    int benefits = 15000;
}

// Driver Class
class Gfg{
    public static void main(String args[]){

        Engineer E1 = new Engineer();
        System.out.println("Salary : " + E1.salary
                           + "\nBenefits : " + E1.benefits);
    }
}

```

Output

```

Salary : 70000
Benefits : 15000

```

15. Java Maths Class

In Java, there exists a Math Library that can simplify intricate calculations

Library:

```
import java.lang.Math;
```

Use:

```
Math.function_name <parameters>
```

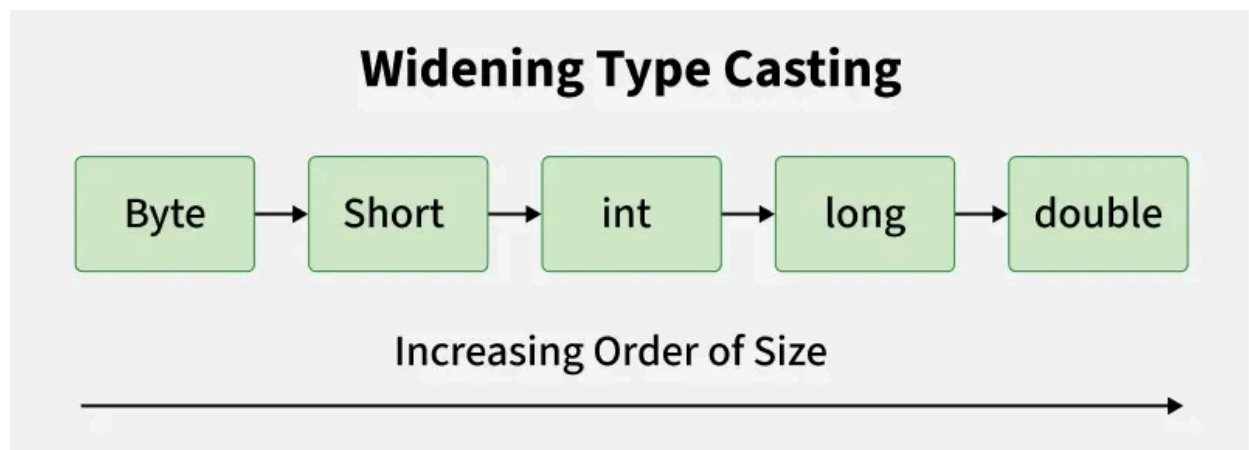
To know more about Maths Class -> [Maths Class](#)

16. Typecasting In Java

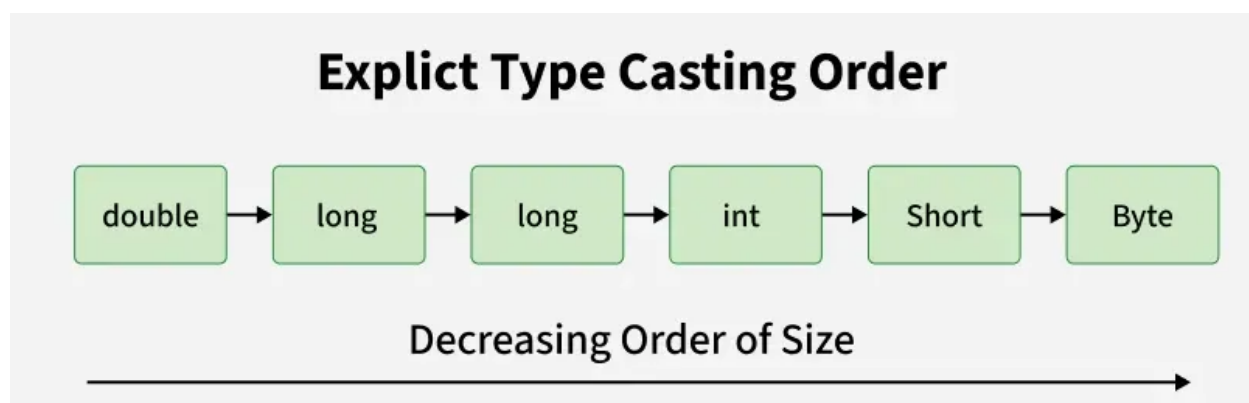
In Java, typecasting is the process of converting one data type to another data type.

here are two types of Type Casting in Java:

1. Widening Type Casting: Converting a smaller data type to a larger data type (automatic).



2. Expli Type Casting: Converting a larger data type to a smaller data type (manual).



Below is the implementation of the above methods:

Java

Java

```
// Java Program to Implement
// Type Casting of the Datatype
import java.io.*;

// Main class
public class GFG {
    // Main driver method
    public static void main(String[] args)
    {
```

× ▷ □

```
int a = 3;

// Casting to Large datatype
double db = (double)a;

// Print and display the casted value
System.out.println(db);
    }
}
```

Output

3.0

17. Arrays in Java

[Arrays](#) are the type of data structure that can store data of similar data types. Arrays are allocated in contiguous memory allocation with a fixed size.

Syntax:

```
// Both Methods are correct
int arr[]=new int[20];
int[] arr=new int[20];
```

Below is the implementation of the above method:

```
class GFG {
    public static void main(String[] args){

        // declares an Array of integers.
        int[] arr = new int[5];

        // Allocating memory to the
        // elements
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;

        // accessing the elements of the specified array
        for (int i = 0; i < arr.length; i++)
            System.out.println("arr[" + i + "] :" + arr[i]);
    }
}
```

Output

```
arr[0] :10
arr[1] :20
```

```
arr[2] :30  
arr[3] :40  
arr[4] :50
```

Multidimensional Arrays in Java

Arrays are not bounded to be single-dimensional but can store elements in multidimensional.

Syntax:

```
int[][] arr= new int[3][3];  
int arr[][]=new int[3][3];
```

Below is the implementation of the above method:

```
public class multiDimensional {  
    // main function  
    public static void main(String args[]){  
  
        // declaring and initializing 2D array  
        int arr[][]  
            = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };  
  
        // printing 2D array  
        for (int i = 0; i < 3; i++){  
  
            for (int j = 0; j < 3; j++){  
                System.out.print(arr[i][j] + " ");  
  
                System.out.println();  
            }  
        }  
    }  
}
```

Output

```
1 2 3  
4 5 6  
7 8 9
```

18. Strings in Java

Strings are the type of objects that can store the character of values. A string acts the same as an array of characters in Java.

Syntax:

```
String abc=" ";
```

CharSequence Interface in Java

1. StringBuffer

A mutable sequence of characters that is thread-safe.

```
StringBuffer s = new StringBuffer("GeeksforGeeks");
```

2. StringBuilder

A mutable sequence of characters that is not thread-safe.

```
StringBuilder str = new StringBuilder();  
str.append("GFG");
```

3. String Tokenizer

A utility to split or join strings using delimiters.

```
public StringJoiner(CharSequence delimiter)
```

19. Java Regex

Regular Expressions (Regex) in Java are used to define patterns for searching, matching, and manipulating strings. They are commonly applied in validations, such as emails or passwords. Java provides the `java.util.regex` package, which includes three main components:

1. Pattern Class

The Pattern class represents a compiled regex pattern. It has no public constructors; patterns are created using the `compile()` method.

Key Methods:

- **`compile(String regex)`**: Compiles a regular expression into a Pattern.
- **`compile(String regex, int flags)`**: Compiles with specific flags.
- **`matcher(CharSequence input)`**: Creates a Matcher to perform match operations on the input.
- **`matches(String regex, CharSequence input)`**: Checks if the input matches the regex.
- **`split(CharSequence input)`**: Splits input around matches.
- **`pattern()`**: Returns the regex string used to create the pattern.
- **`quote(String s)`**: Escapes special characters in a string.

2. Matcher Class

The Matcher class is used to perform match operations on a string using a Pattern.

Key Methods:

- **`find()`**: Searches for the next occurrence of the pattern.

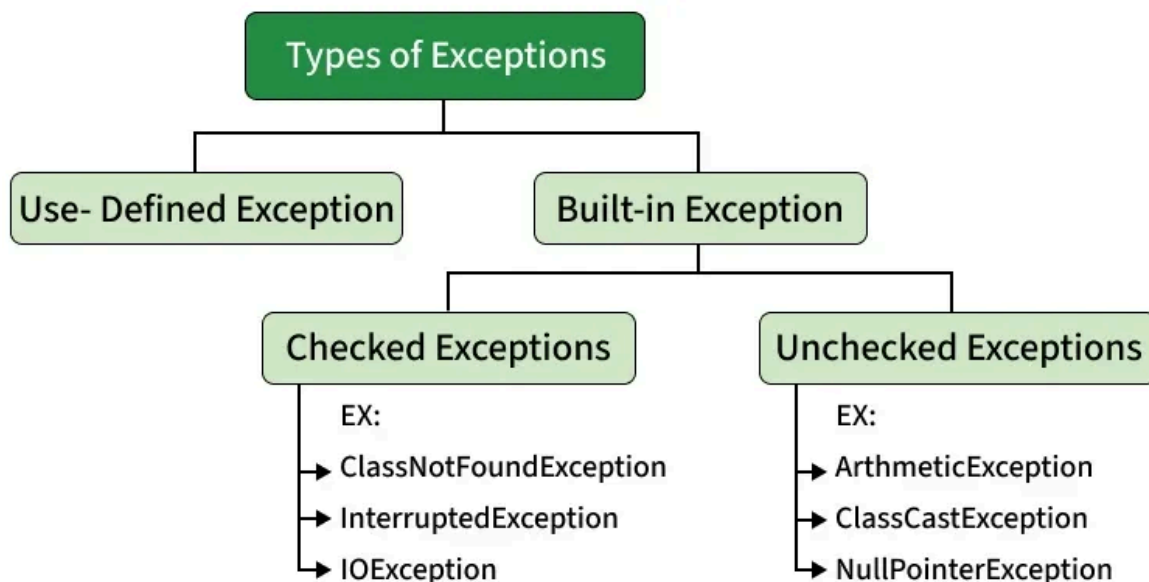
- **find(int start):** Searches starting from a specific index.
- **start():** Returns the start index of the current match.
- **end():** Returns the end index of the current match.
- **groupCount():** Returns the number of matched groups.
- **matches():** Checks if the entire input matches the pattern.

20. Java Exception Handling

An Exception is an unexpected error that occurs during the run-time of a program.

Types of Exceptions

- **Checked Exception:** This mainly includes IO Exceptions and Compile time Exceptions
- **Unchecked Exceptions:** This covers both Runtime Exceptions and Null Pointer Exceptions.



Handling Exceptions

- **try block:** Contains code that might throw an exception.
- **catch block:** Handles exceptions thrown in the try block. Multiple catch blocks can handle different exception types.
- **finally block:** Always executes after try/catch blocks, used for cleanup like closing files or releasing resources

Syntax:

```

try {
    // code that may throw an exception
} catch (ExceptionType1 ex) {
    // handle ExceptionType1
} catch (ExceptionType2 ex) {

```

```
// handle ExceptionType2
} finally {
    // code executed regardless of exception
}
```

Final vs finally vs finalize

- **final:** Used with classes, methods, and variables. A final class cannot be inherited, a final method cannot be overridden, and a final variable cannot be reassigned.
- **finally:** A block that always executes after try/catch, typically used to release resources.
- **finalize():** Called by the garbage collector before object destruction for cleanup; not guaranteed to run.

throw vs throws

- **throw:** Used to explicitly throw an exception from a method or block. Can throw checked or unchecked exceptions.
- **throws:** Declares exceptions a method might throw, useful when no try/catch is present.

21. Java Commands

Here are the most commonly used Java commands:

- **java -version:** Displays the version of the Java Runtime Environment (JRE) that is installed on your computer.
- **java -help:** Displays a list of all of the Java commands.
- **java -cp:** Specifies the classpath for the Java program that you are running.
- **java -jar:** Runs a Java Archive (JAR) file.
- **java -D:** Sets a system property.
- **java -X:** Specifies a non-standard option for the Java Virtual Machine (JVM).

Here are some other useful Java commands:

- **javac:** Compiles a Java source file into bytecode.
- **javap:** Disassembles a Java class file.
- **jdb:** A Java debugger.
- **jconsole:** A Java monitoring and management tool.
- **jvisualvm:** A Java profiling and diagnostic tool.

22. Java Generics

Generics in Java allow classes, interfaces, and methods to operate on parameterized types (like Integer, String, or user-defined types). This enables type safety and eliminates the need for explicit type casting.

Key Concepts

- A generic entity is a class, interface, or method that works with parameterized types.
- Generics allow creating reusable code that works with different data types.

Types of Java Generics

1. Generic Class: A class that can work with different types defined at instantiation.

```
BaseType<Type> obj = new BaseType<Type>();
```

2. Generic Method: A method with type parameters that can accept and return different types. The compiler ensures type safety.

```
public <T> T genericMethod(T input) {  
    return input;  
}
```

3. Generic Function: A function designed to operate with various types of arguments. The compiler automatically handles type checks based on the argument types.

```
class Test {  
    // A Generic method example  
    static <T> void genericDisplay(T element){  
  
        System.out.println(element.getClass().getName()  
                               + " = " + element);  
    }  
  
    // Driver method  
    public static void main(String[] args){  
  
        // Calling generic method with Integer argument  
        genericDisplay(11);  
  
        // Calling generic method with String argument  
        genericDisplay("GeeksForGeeks");  
  
        // Calling generic method with double argument  
        genericDisplay(1.0);  
    }  
}
```

Output

```
java.lang.Integer = 11  
java.lang.String = GeeksForGeeks  
java.lang.Double = 1.0
```

23. Java Multithreading

Multithreading is a Java feature that allows concurrent execution of two or more parts of a program to maximize CPU utilization. Each part of a program executed concurrently is called a thread, which is a lightweight process within a process.

Creating Threads in Java

Threads can be created using two main approaches:

1. **Extending the Thread class**
2. **Implementing the Runnable interface**

Example: Creating a Thread by Extending Thread Class

To create a thread by extending the Thread class:

1. Extend the java.lang.Thread class.
2. Override the run() method – this defines the code to be executed by the thread.
3. Create an instance of your class.
4. Call the start() method to begin execution. The start() method internally calls the run() method.

```
class MultithreadingDemo extends Thread {  
    public void run()  
    {  
        try {  
            // Displaying the thread that is running  
            System.out.println(  
                "Thread " + Thread.currentThread().getId()  
                + " is running");  
        }  
        catch (Exception e) {  
            // Throwing an exception  
            System.out.println("Exception is caught");  
        }  
    }  
}  
  
// Main Class  
public class Multithread {  
    public static void main(String[] args){  
  
        int n = 8; // Number of threads  
        for (int i = 0; i < n; i++){  
  
            MultithreadingDemo object  
                = new MultithreadingDemo();  
            object.start();  
        }  
    }  
}
```

}

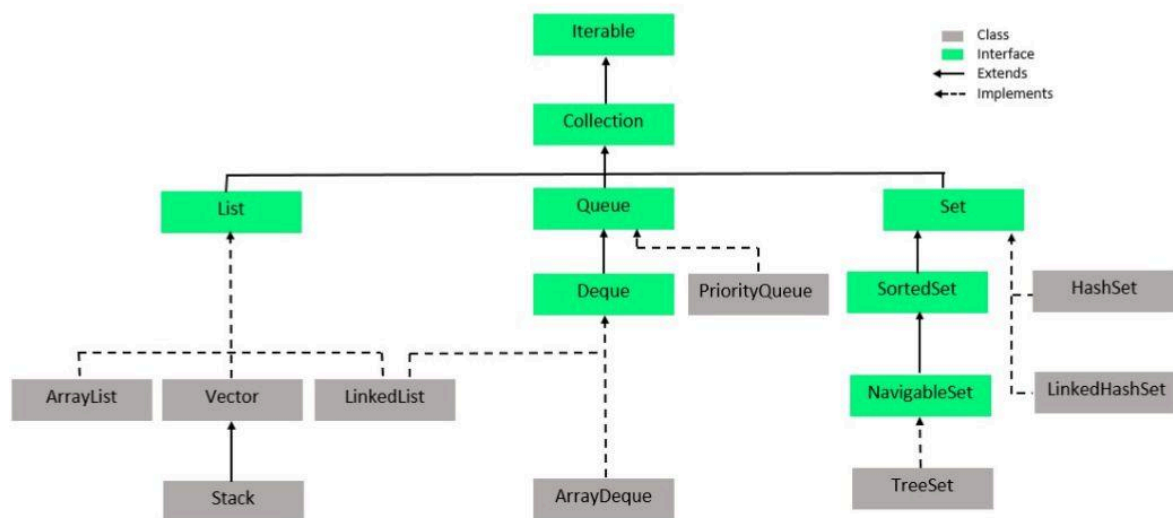
Output

```
Thread 15 is running
Thread 17 is running
Thread 14 is running
Thread 12 is running
Thread 13 is running
Thread 18 is running
Thread 11 is running
Thread 16 is running
```

24. Java Collections

The [collection is a framework](#) in Java that provides an architecture to store and manipulate objects in Java. It contains the interface as mentioned below:

- List Interface
- Queue Interface
- Deque Interface
- Set Interface
- SortedSet Interface
- Map Interface
- Collection Interface
- Iterable Interface



collection-framework

Why Use Java?

Java is simple for programmers to learn. Java is frequently chosen as the first programming language to learn. Furthermore, the sector continues to benefit from Java's prominence. The majority of websites and apps in the government, healthcare, defence, and educational sectors continue to use Java technology. Java is thus worthwhile to learn and use. If you choose a career in Java, you can follow a number of different career routes. Almost anything that Java can accomplish.

Applications of Java Programming language

- Mobile Applications
- Desktop GUI Applications
- Artificial intelligence
- Scientific Applications
- Cloud Applications
- Embedded Systems
- Gaming Applications

[Comment](#)

GeeksforGeeks

86



Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

[About Us](#)[Legal](#)[Privacy](#)[Policy](#)[Careers](#)[Contact Us](#)[Corporate](#)[Solution](#)[Campus](#)[Training](#)

Explore

[POTD](#)[Practice](#)[Problems](#)[Blogs](#)[Upskill](#)[Courses](#)[Connect](#)

Tutorials

[Programming](#)[Languages](#)[DSA](#)[Web](#)[Technology](#)[AI, ML &](#)[Data Science](#)[DevOps](#)[CS Core](#)[Subjects](#)[GATE](#)[School](#)[Subjects](#)[Software and](#)[Tools](#)

Courses

[ML and Data](#)[Science](#)[DSA and](#)[Placements](#)[Web](#)[Development](#)[Data Science](#)[Programming](#)[Languages](#)[DevOps &](#)[Cloud](#)[GATE](#)[MongoDB](#)[Certifications](#)

Preparation Corner

[Interview](#)[Corner](#)[Aptitude](#)[Puzzles](#)[GfG 160](#)[System Design](#)

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved